TCPU-16

16-bit Accumulator, 8-bit Instruction, Single-Cycle CPU

Micro-architecture Specification

Document ID	TCPU16-SPEC-001
Revision	1.0, 15 October 2026. First release.
Audience	RTL designer. This document is sufficient to implement and verify the design.
Target	SkyWater SKY130 open MPW shuttle, standard-cell digital flow, Verilog/ SystemVerilog RTL.
Designer-declared values	Clock period, clocking mode, PVT corners, harness and pin mapping, area allocation. This document does not fix them; the designer selects each one and records it in the design note (10.2). Verification checks against the declared values.

Contents

1. Introduction 2
2. Design requirements and constraints 3
3. Top-level interface 5
4. Micro-architecture 6
5. Programmer's model and register / memory maps 7
6. Instruction set architecture 8
7. Functional specification 10
8. Worked examples: programs 14
9. Features to be verified 16
10. Implementation guidance, deliverables and acceptance 17
11. Programming model, out of scope and risks 18

1. Introduction

1.1 Purpose and scope

TCPU-16 is a single-cycle accumulator CPU with a 16-bit accumulator, 8-bit instructions and an 8-word program memory. One instruction is fetched, decoded, executed and committed at each rising clock edge. Only the accumulator is 16 bits wide; the instruction word and the program memory are 8 bits wide. The program is loaded through input ports before execution. There is no data RAM, no register file and no interrupt support.

This document defines the externally visible behaviour (interface, instruction set, edge-level timing) and the required micro-architecture. **Shall** marks a mandatory requirement, checked by the verification plan (Section 9). **Should** marks recommended practice. **Designer-declared** marks a value this document deliberately leaves open; the designer selects it and records it in the design note (10.2).

1.2 Conventions

- Numbers: `0x` prefix hexadecimal, `0b` binary, otherwise decimal. Bit ranges are written [msb:lsb].
- *Edge* means the rising edge of `clk`. Cycle *n* is the *n*-th edge with run=1 since execution started (the edge that executes the instruction at PC=0 is cycle 1).
- All timing in this document is expressed in clock edges. No absolute frequency or period is specified (2.3).
- All register updates described for an edge use the values present before that edge (non-blocking semantics).
- `rst_n` is active low. Signal names are shown in monospace.

1.3 Glossary

Term	Meaning
ACC	Accumulator: the only general-purpose data register (16 bits).
PC	Program counter: 3-bit address of the instruction executed at the next edge.
IR	Instruction register: copy of the most recently committed instruction (RTL simulation visibility only, 4.2).
Opcode / operand	Bits [7:5] / [4:0] of an instruction word.
Immediate (imm)	The 5-bit operand of LOAD/ADD/SUB/XOR, zero-extended to 16 bits.
Shift amount (shamt)	operand[2:0] of SHL: number of bit positions ACC is shifted left (0 to 7).
Target	The jump destination: operand[2:0] of JMP/JZ.
Commit	The edge at which an instruction's results are written to the architectural state.
Low / high byte	ACC[7:0] / ACC[15:8].
Host	The external controller (a microcontroller board or a testbench) that drives the input ports and reads `acc_out`.
Clocking mode	Mode H (host-generated clock) or Mode O (independent clock source), declared by the designer (2.1).
SYNC_STAGES	Number of synchronizer flops on `run`, `prog_load` and `rst_n`: 0 in Mode H, 2 in Mode O.
Design note	The deliverable that records every designer-declared value and every deviation (10.2).

1.4 Key parameters

Data width / instruction width	16 bits (ACC) / 8 bits
Instruction format	3-bit opcode + 5-bit operand
Program memory	8 words x 8 bits, flip-flop array, loaded through ports, read-only while running

Program counter	3 bits, wraps 7 → 0
Opcodes	8: LOAD, ADD, SUB, SHL, XOR, JMP, JZ, HALT
Output	8-bit port `acc_out`: one byte of ACC, selected by `byte_sel`
Throughput	1 instruction per edge (CPI = 1), no pipeline, no stalls
Clock frequency	Designer-declared (2.3). Behaviour is defined per edge and does not depend on frequency.
Clocking mode / SYNC_STAGES	Designer-declared: Mode H (SYNC_STAGES=0) or Mode O (SYNC_STAGES=2) (2.1)
Reset	Synchronous, active low (`rst_n`)
Fixed parameters	PROG_WORDS=8, OPCODE_BITS=3, OPERAND_BITS=5, ACC_WIDTH=16 (not configurable)

2. Design requirements and constraints

2.1 Clock, clocking mode, reset and input timing

- **Single clock domain.** All state elements are clocked on the rising edge of `clk`. No gated or derived clocks, no negative-edge logic.
- **Clock frequency.** Not specified by this document. The designer selects a target clock period, closes timing against it (2.3) and records it with the reason for the choice.
- **Clocking mode.** The designer shall declare one of the two modes below and set SYNC_STAGES accordingly.

Mode	Condition	Requirement
H: host-generated clock	The host generates `clk` and changes `run`, `prog_load`, `prog_data` and `rst_n` only while `clk` is low, settled before the next rising edge.	SYNC_STAGES = 0. Inputs are synchronous to `clk` by construction; no synchronizers are present.
O: independent clock	`clk` comes from an oscillator or harness clock that the host does not control. Host inputs are asynchronous to `clk`.	SYNC_STAGES = 2. `run`, `prog_load` and `rst_n` each pass through a two-flop synchronizer before any other use. `prog_data` is not synchronized; the load protocol (7.2) keeps it stable while it is sampled.

> Why Mode O needs synchronizers: an asynchronous `run` sampled directly can be resolved as 1 by some of the PC, ACC, IR and halted flops and as 0 by others at the same edge, committing part of an instruction (for example PC advances while ACC does not). This is a functional failure, not only a metastability risk.

- **Synchronized signal names.** In Mode O, the names `run`, `prog_load` and `rst_n` in Sections 4, 7 and 9 denote the synchronizer outputs. The pins lead these by SYNC_STAGES edges; all edge counts in this document refer to the synchronized signals.
- **Reset.** `rst_n` is sampled synchronously. The host shall hold `rst_n`=0 for at least SYNC_STAGES + 1 edges. Reset has priority over every other action, except that `pl_q` samples `prog_load` at every edge, including during reset (7.3).
- **`byte_sel`** is not sampled by any flip-flop in either mode. It only steers the output multiplexer and may change at any time; the host reads `acc_out` after the multiplexer has settled (7.4).

2.2 Implementation constraints

- Fully synchronous RTL. No latches, no tri-states, no combinational loops, no asynchronous reset.
- Permitted datapath operators: 16-bit addition, 16-bit subtraction, bitwise XOR, logical left shift by a 3-bit amount, 16-bit compare-with-zero and multiplexers. No multiplication, division, modulo or variable right shift.

- No RAM macro: program memory is an array of flip-flops with no reset (smaller cells). Its content is undefined until written.
- The ports of `toy_cpu16` are plain inputs and outputs (3.1). A harness wrapper may connect them to bidirectional pads whose direction is tied permanently (2.4).
- Control is a combinational decoder plus the single `halted` flag. There is no multi-cycle controller state machine.

2.3 Timing closure (designer-declared)

Static timing analysis in the implementation flow shall show setup and hold slack ≥ 0 at the designer-declared clock period and at every designer-declared PVT corner. No minimum frequency is required. The design note shall record:

1. The clock period and the reason for the choice (for example the host's clock toggle rate in Mode H, or the harness clock in Mode O).
2. The PVT corners analysed: at least one setup-critical (slow) corner and one hold-critical (fast) corner of the SKY130 standard-cell library used.
3. Input and output delay constraints for every port relative to `clk`, and the clock uncertainty and transition assumptions.
4. The treatment of the combinational path `byte_sel` → `acc_out`: either a max-delay constraint or a documented false path relative to `clk`.
5. Worst setup and hold slack per corner.
6. Mode O only: the paths from the `run`, `prog_load` and `rst_n` pins to their first synchronizer flop, and from `prog_data` to the program memory, are asynchronous and shall be declared as false paths. `prog_data` is safe because 7.2 holds it stable for at least 2 edges before the write edge.

The timing report shall cover these paths:

#	Path	Note
1	`pc` → program-memory read mux → decode → 16-bit adder/subtractor or shifter → ACC mux → `acc`	Expected critical path
2	`acc` → 16-bit zero detect → next-PC mux → `pc`	JZ
3	`acc` → byte-select mux → `acc_out`	Register-to-output
4	`byte_sel` → `acc_out`	Input-to-output, no register
5	`prog_load`/`run` → `load_pulse` → memory write enable	Input-to-register (Mode H)

2.4 Target harness, pins and area (designer-declared)

`toy_cpu16` needs 13 input pins (`clk`, `rst_n`, `prog_data[7:0]`, `prog_load`, `run`, `byte_sel`) and 8 output pins. The design note shall name the shuttle or harness, map every port to a harness pin and state the area allocation for the block.

- If the harness has fewer than 13 dedicated input pins, the remaining inputs use harness bidirectional pins with their output enable tied off in the wrapper. For example, on a Tiny Tapeout tile `prog_data` maps to `ui_in[7:0]`, `prog_load`/`run`/`byte_sel` to `uio_in[2:0]` with `uio_oe` = 0, and `acc_out` to `uo_out[7:0]`.
- Utilisation after synthesis and after placement shall be reported against the allocation. If it exceeds the allocation, apply the levers of 4.3 in the order listed.

3. Top-level interface

3.1 Ports

The top module shall be named `toy_cpu16` and have exactly the following ports:

Port	Dir	Width	Description
`clk`	in	1	Clock; all state is clocked on the rising edge
`rst_n`	in	1	Synchronous active-low reset
`prog_data`	in	8	Instruction byte to be loaded into program memory
`prog_load`	in	1	Load strobe, rising-edge detected
`run`	in	1	Run enable
`byte_sel`	in	1	Selects which half of ACC is visible on `acc_out`
`acc_out`	out	8	Selected accumulator byte, continuously driven

3.2 Signal descriptions

Signal	Sense	Description
`prog_data[7:0]`	level	Byte written to program memory when a load pulse is detected. Shall be set no later than `prog_load` rises and held until `prog_load` falls (7.2).
`prog_load`	rising-edge detected	A 0 → 1 transition seen at an edge (previous sample 0, current sample 1) while run=0 writes `prog_data` to `mem[load_addr]` and increments `load_addr`. Holding the signal high does not cause repeated writes. The detector does not debounce: `prog_load` shall be driven by logic (microcontroller GPIO or testbench), not by a mechanical switch.
`run`	level	1 = execute one instruction per edge (unless halted). 0 = pause: all architectural state holds; program loading is allowed.
`byte_sel`	level, combinational	0: `acc_out` = ACC[7:0]. 1: `acc_out` = ACC[15:8]. Takes effect without a clock edge.
`acc_out[7:0]`	combinational	`byte_sel ? ACC[15:8] : ACC[7:0]`, driven from the live accumulator. Changes when ACC changes (at a committing edge or on reset) or when `byte_sel` changes.

There is no `step` port. Single-stepping is done by pulsing `clk` from the host with run=1 (Mode H).

3.3 Verification visibility (mandatory names)

The `toy_cpu16` module scope shall contain registers with exactly these names and widths: `pc[2:0]`, `acc[15:0]`, `ir[7:0]`, `halted`. All four are mandatory in RTL; the testbench reads them hierarchically. They are not ports. If the design uses submodules, these four registers shall still be declared in the `toy_cpu16` scope.

In the gate-level netlist these names are not preserved: `ir` is removed by synthesis and the others are split into per-bit flops. Gate-level checks therefore use `acc_out` only (N4).

4. Micro-architecture

4.1 Block diagram

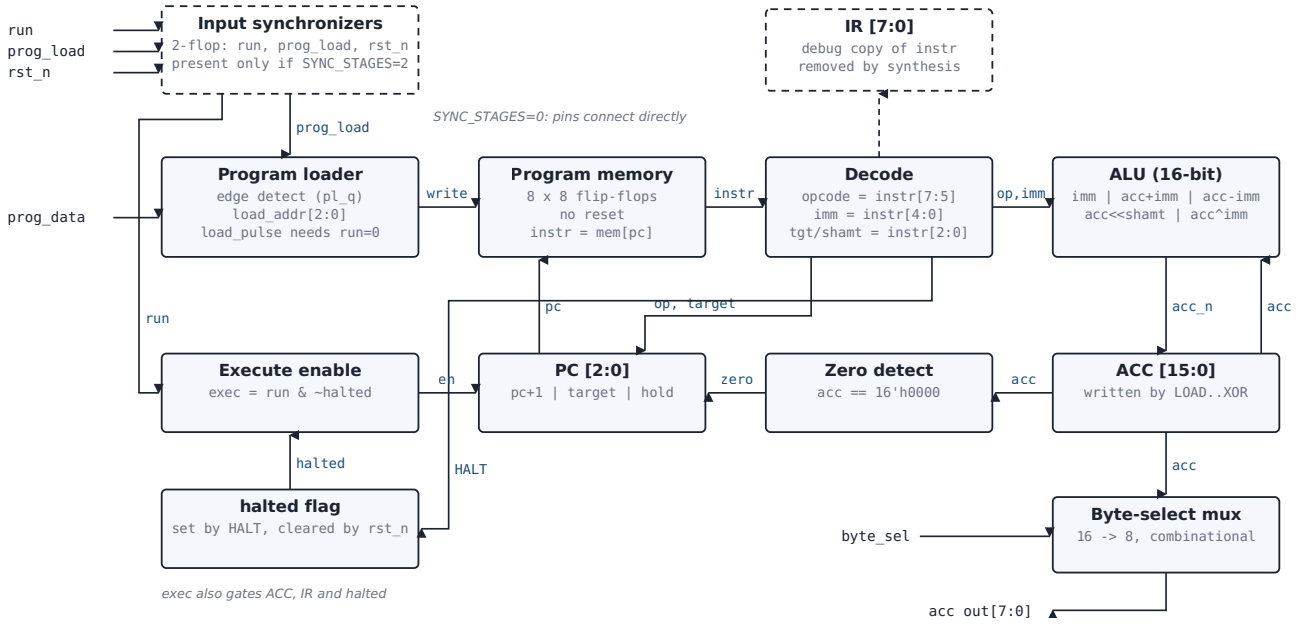


Figure 1: TCPU-16 block diagram. Solid boxes are synthesized logic. The dashed synchronizer block exists only when SYNC_STAGES=2. The dashed IR has no fanout and is removed by synthesis. `exec` gates the PC, ACC, IR and halted updates; the loader and the byte-select mux are independent of `exec`.

4.2 Block descriptions

Block	Function and requirements
Input synchronizers	Present only when SYNC_STAGES=2. Two flops in series for each of `run`, `prog_load` and `rst_n`, no reset, no logic between the two flops. All other logic uses the second flop's output.
Program memory	8 × 8-bit register array. Combinational read: `instr = mem[pc]`. Written only by the loader. No reset; contents persist across `rst_n`.
Program loader	Registers `pl_q <= prog_load` at every edge, including during reset. `load_pulse = prog_load & ~pl_q & ~run`. On `load_pulse`: `mem[load_addr] <= prog_data`; `load_addr <= load_addr + 1` (3-bit, wraps 7 → 0). `load_addr` is cleared by reset.
Execute enable	`exec = run & ~halted`. When `exec`=0, PC, ACC, IR and halted hold their values.
Decode	Splits `instr` into `opcode = instr[7:5]`, `imm = {11'b0, instr[4:0]}` (16 bits), `target = shamt = instr[2:0]`. Generates ALU-select, PC-select and halt controls. `instr[4:3]` is ignored by SHL, JMP and JZ.
ALU	16-bit. Selects: pass imm (LOAD), acc + imm (ADD), acc – imm (SUB), acc << shamt (SHL, logical, bits shifted out are lost), acc ^ imm (XOR); otherwise ACC holds. Modulo-65536 result, no flags. The carry or borrow of ADD/SUB propagates through all 16 bits.
Zero detect	`acc == 16'h0000` over all 16 bits, evaluated on the pre-edge ACC. Used only by JZ.
PC and next-PC	`pc_n = pc + 1` by default; `target` for JMP, and for JZ when acc == 0; `pc` (hold) for HALT.
ACC	16 bits. Written by LOAD, ADD, SUB, SHL and XOR only.
Byte-select mux	`acc_out = byte_sel ? acc[15:8] : acc[7:0]`. Purely combinational: no register between ACC and `acc_out`.
IR	`ir <= instr` at every committing edge. Exists for RTL simulation visibility only. It has no logic fanout, so synthesis removes it; it shall not be connected to any output to preserve it.
Halted flag	Set when HALT commits. Cleared only by `rst_n`. While set, `exec` = 0.

4.3 State and area budget

State element	Bits	Reset	Flops in synthesized netlist
Program memory	64	none	64
`acc`	16	0x0000	16
`pc`	3	0	3
`load_addr`	3	0	3
`halted`	1	0	1
`pl_q`	1	none (tracks `prog_load`)	1
`ir`	8	0x00	0 (removed)
Input synchronizers (Mode O only)	6	none	6
Total	**96 in RTL (+6 in Mode O)**		**88 required (94 in Mode O)**

Flop-count acceptance. The synthesized flop count shall be 88 (94 in Mode O). Generic Yosys synthesis with the default `synth -flatten` reports 91 for the reference RTL: the memory pass merges `pc` into the program-memory read port as a read-address register and keeps the original `pc` for the next-PC logic, so the netlist contains a 3-flop duplicate of `pc`. `synth -flatten -nordff` prevents the merge and reports exactly 88. A count of 91 (97 in Mode O) is accepted only if the design note identifies the 3 extra flops as this duplicate. Any other extra flop fails N2.

Area reference (indicative only). Generic Yosys synthesis of a reference RTL (`synth -flatten -noabc -nordff`, not technology-mapped) gives 513 cells, 88 flops and no latches. Removing SHL saves 86 cells, so the shifter is about 17 % of the logic. The authoritative figure is the post-synthesis report of the implementation flow, compared with the area allocation (2.4). Cell counts change between Yosys versions; the design note records the version used (10.2).

Lever if the design does not fit	Measured effect (generic synthesis, reference RTL)	Comment
1. Reduce PROG_WORDS from 8 to 4	513 → 425 cells (−88); flops 88 → 54	Largest lever. `pc`, `load_addr` and the jump target become 2 bits (target = operand[1:0]); program length halves.
2. Remove XOR	513 → 491 cells (−22)	XOR is never required to reach a value.
Remove SHL (not recommended)	513 → 427 cells (−86)	Without SHL only 435 distinct values are reachable within 7 instructions (6.6), so the ISA loses practical access to 16-bit values.

All counts: `synth -flatten -noabc -nordff` on modified copies of the reference RTL. Apply the levers in the order shown. Any lever applied is a deviation and shall be listed in the design note.

5. Programmer's model and register / memory maps

There are no memory-mapped control/status registers. The CPU has no bus and no address-decoded register space. Control and status use dedicated ports (5.3); the only addressable resource is the program memory (5.2). The tables below are the complete map.

5.1 Architectural state

Register	Width	Reset	Written by	Observable via
ACC	16	0x0000	LOAD, ADD, SUB, SHL, XOR	`acc_out` (one byte at a time); `acc` in simulation
PC	3	0	every committed instruction	`pc` in simulation
IR	8	0x00	every committed instruction	`ir` in RTL simulation only

Register	Width	Reset	Written by	Observable via
halted	1	0	HALT	`halted` in simulation; on hardware, ACC stops changing
load_addr	3	0	load pulse	none (internal to loader)

5.2 Program memory map

Address	Access	Contents
0 – 7	Write: sequentially by load pulses, only while run=0 (address = `load_addr`). Read: by PC during execution only.	One 8-bit instruction per address. Not cleared by reset. Undefined until loaded: all 8 words shall be written before `run` is asserted. There is no readback path; a mis-loaded word is detectable only by executing a program (11.3).

5.3 Port-level control and status map

Function	Port	Type	Reset / idle state
Program data	`prog_data[7:0]`	input	don't care
Load strobe	`prog_load`	input, edge	0
Run enable	`run`	input, level	0
Byte select	`byte_sel`	input, level	0
Accumulator byte	`acc_out[7:0]`	output	0x00 after reset (both halves 0)

byte_sel	acc_out	Read order
0	ACC[7:0] (low byte)	Step 1
1	ACC[15:8] (high byte)	Step 2

6. Instruction set architecture

6.1 Instruction format

Bit	7	6	5	4	3	2	1	0
Field	opcode[2:0]			operand[4:0]				

The operand is an immediate (imm = operand[4:0], 0 to 31, zero-extended to 16 bits) for LOAD/ADD/SUB/XOR, a shift amount (shamt = operand[2:0], 0 to 7) for SHL, and a jump target (target = operand[2:0]) for JMP/JZ. HALT ignores the operand.

6.2 Instruction set

Opcode	Mnemonic	Byte range	Operation (executed at the edge)	PC after
000	LOAD imm	0x00–0x1F	`ACC <= {11'b0, imm}`	PC + 1
001	ADD imm	0x20–0x3F	`ACC <= (ACC + imm) mod 65536`	PC + 1
010	SUB imm	0x40–0x5F	`ACC <= (ACC − imm) mod 65536`	PC + 1
011	SHL sh	0x60–0x7F	`ACC <= (ACC << sh) mod 65536`, sh = operand[2:0]	PC + 1

Opcode	Mnemonic	Byte range	Operation (executed at the edge)	PC after
100	XOR imm	0x80–0x9F	`ACC <= ACC ^ {11'b0, imm}`	PC + 1
101	JMP addr	0xA0–0xBF	none	addr (= operand[2:0])
110	JZ addr	0xC0–0xDF	none	addr if ACC == 0 (all 16 bits), else PC + 1
111	HALT	0xE0–0xFF	`halted <= 1`	PC (held)

6.3 Semantics

- **Arithmetic.** ADD and SUB operate modulo 65536 on all 16 bits; a carry out of bit 7 enters bit 8, and a borrow likewise. There is no carry, borrow, overflow or sign flag. Immediates are unsigned.
- **LOAD** replaces all 16 bits: the upper 11 bits become 0.
- **SHL.** Logical shift left by operand[2:0]. Zeros enter at bit 0; bits shifted out of bit 15 are lost (no rotate, no carry). SHL 0 commits (PC and IR advance) and leaves ACC unchanged.
- **XOR.** The 5-bit immediate is zero-extended, so XOR changes only ACC bits 4:0. XOR 0 commits and leaves ACC unchanged.
- **JZ.** The condition is ACC == 0 over all 16 bits, evaluated on ACC before the current edge, i.e. the result of the preceding instruction. A SUB immediately followed by JZ tests the SUB result. 0x0100, whose low byte is zero, is not zero.
- **HALT.** At the HALT edge IR takes the HALT word, `halted` is set and PC holds. After that PC, ACC, IR and `halted` do not change until `rst_n`. Program memory and `load_addr` can still be written while run=0 (7.3).
- **Ignored fields.** Hardware ignores operand[4:3] for SHL, JMP and JZ and the whole operand for HALT. Programs should write zeros there; verification includes programs that do not (F18, F25).
- **No output instruction.** ACC is always visible on `acc_out`, so there is no OUT or STORE.

6.4 Ten worked examples

The ten examples cover every opcode, both JZ outcomes, and the 16-bit behaviours: carry and borrow across the byte boundary, bits lost by SHL, zero-extension. "Before" is the state at the start of the cycle, "After" the state after the committing edge. Unlisted state is unchanged.

#	Instruction	Encoding	Before	After	Explanation
1	LOAD 21	000_10101 = 0x15	ACC=0xABCD, PC=0	ACC=0x0015, PC=1	All 16 bits replaced by the zero-extended immediate; the old high byte 0xAB is cleared.
2	ADD 5	001_00101 = 0x25	ACC=0x00FE, PC=1	ACC=0x0103, PC=2	0x00FE + 0x0005 = 0x0103. The carry out of the low byte enters the high byte.
3	SUB 4	010_00100 = 0x44	ACC=0x0100, PC=2	ACC=0x00FC, PC=3	0x0100 − 0x0004 = 0x00FC. The borrow comes from the high byte.
4	SHL 4	011_00100 = 0x64	ACC=0x0012, PC=3	ACC=0x0120, PC=4	0x0012 << 4 = 0x0120.
5	SHL 7	011_00111 = 0x67	ACC=0xFF01, PC=4	ACC=0x8080, PC=5	0xFF01 << 7 = 0x7F8080; the low 16 bits are kept. Bits shifted out of bit 15 are lost.
6	XOR 21	100_10101 = 0x95	ACC=0xFFFF, PC=5	ACC=0xFFEA, PC=6	Immediate zero-extended (0x0015); only bits 4:0 flip.
7	JMP 6	101_00110 = 0xA6	PC=3, ACC=0x0007	PC=6, ACC=0x0007	PC takes operand[2:0] = 6. ACC unchanged.
8	JZ 4 (taken)	110_00100 = 0xC4	PC=2, ACC=0x0000	PC=4	All 16 bits are zero, so PC takes the target.

#	Instruction	Encoding	Before	After	Explanation
9	JZ 4 (not taken)	110_00100 = 0xC4	PC=2, ACC=0x0100	PC=3	Low byte 0x00, high byte 0x01: ACC is non-zero. A low-byte-only comparator would wrongly branch.
10	HALT	111_00000 = 0xE0	PC=5, halted=0	PC=5, halted=1	PC, ACC, IR and halted do not change at later edges, even if `run` is toggled; only `rst_n` clears `halted`. Program memory can still be loaded while run=0.

6.5 Wrap-around examples

Instruction	Before	After	Explanation
ADD 31 (0x3F)	ACC=0xFFF0	ACC=0x000F	0xFFF0 + 0x001F = 0x1000F; bit 16 is discarded. No flag.
SUB 31 (0x5F)	ACC=0x0003	ACC=0xFFE4	3 − 31 = −28 = 0xFFE4 modulo 65536. No flag.

6.6 Building a 16-bit constant

One instruction supplies a 5-bit immediate (0 to 31). Wider constants are built from several instructions. The basic recipe uses three:

- **LOAD a** (a = 0 to 31), **SHL s** (s = 0 to 7), **ADD b** (b = 0 to 31). The result is exactly (a << s) + b.
- Example: 0x0123 = 291 = (18 << 4) + 3, so LOAD 18 (0x12), SHL 4 (0x64), ADD 3 (0x23).
- The shifted chunk and b may overlap in bit position; the result is still exact arithmetic. For a unique choice of (a, s, b), use $b < 2^s$.
- The recipe reaches 2048 distinct values, the largest being 3999 (31 × 128 + 31).

Exhaustive search over all programs made of LOAD, ADD, SUB, SHL and XOR, starting from the reset state (ACC = 0):

Instructions	New values	Cumulative values reachable
1	62	63
2	254	317
3	5887	6204
4	16894	23098
5	42438	65536 (all)

- Every 16-bit value needs at most 5 instructions.
- 1 instruction: 0 to 31 (LOAD) and 0xFFE1 to 0xFFFF (SUB 1..31 from the reset value 0).
- Values reachable in 2 instructions include every $a \times 2^k$ with $a \le 31$ and $k \le 7$ (LOAD a, SHL k).
- Examples: 0x0123 and 0x8000 need 3; 50000 (0xC350) needs 4.
- Simple method without search: LOAD the top non-zero 5-bit chunk, then repeat SHL 5 and ADD chunk. At most 7 instructions.
- Program memory holds 8 words. A 5-instruction constant plus HALT leaves 2 words for other instructions.
- Every constant-building sequence shall be checked in RTL simulation before it is used on hardware.

7. Functional specification

7.1 Execution model

Execution is single-cycle. Within one clock period `mem[pc]` is read, decoded and its result computed combinationally; at the next edge all results commit together. No instruction takes more than one edge. The behaviour at every edge is defined by the following reference algorithm (right-hand sides use pre-edge values).

This algorithm is the reference model for F35. In Mode O, `run`, `prog_load` and `rst_n` below are the synchronizer outputs.

```
pl_q <= prog_load                       // every edge, including reset
if (!rst_n) {                           // reset has priority
    acc <= 0; pc <= 0; ir <= 0; halted <= 0; load_addr <= 0
    // mem[] is NOT modified
} else {
    load_pulse = prog_load && !pl_q && !run
    if (load_pulse) { mem[load_addr] <= prog_data; load_addr <= load_addr + 1 }  // mod 8
    if (run && !halted) {
        instr = mem[pc]; op = instr[7:5]; imm = instr[4:0]; tgt = sh = instr[2:0]
        ir <= instr; pc <= pc + 1                     // defaults
        switch (op) {
          LOAD : acc <= imm                           // upper 11 bits cleared
          ADD  : acc <= acc + imm                     // 16-bit wrap
          SUB  : acc <= acc - imm                     // 16-bit wrap
          SHL  : acc <= acc << sh                     // bits shifted out are lost
          XOR  : acc <= acc ^ imm                     // imm zero-extended
          JMP  : pc  <= tgt
          JZ   : if (acc == 0) pc <= tgt              // all 16 bits
          HALT : pc  <= pc; halted <= 1
        }
    }
}
acc_out = byte_sel ? acc[15:8] : acc[7:0]              // combinational, always
```

`load_pulse` requires run=0, so loading and execution never occur at the same edge.

7.2 Program loading procedure

Let N = SYNC_STAGES + 1 (1 in Mode H, 3 in Mode O).

1. Hold `rst_n`=0 for at least N edges with run=0, then release it. Keep run=0.
2. For each word w0..w7: set `prog_data` = w no later than raising `prog_load`; hold `prog_load` high for at least N edges; lower it and hold it low for at least N edges. Keep `prog_data` stable for the whole time `prog_load` is high. The word is written at the first edge at which the (synchronized) `prog_load` is sampled as 1.
3. A pulse that is high for several edges writes exactly one word.
4. After 8 writes `load_addr` has wrapped to 0; a 9th write overwrites word 0. Reset returns `load_addr` to 0 without clearing memory.
5. Raise `run` only after `prog_load` has been low for at least N edges following the last word. A load pulse that coincides with run=1 is not written. PC is 0 after reset, so execution begins at word 0.

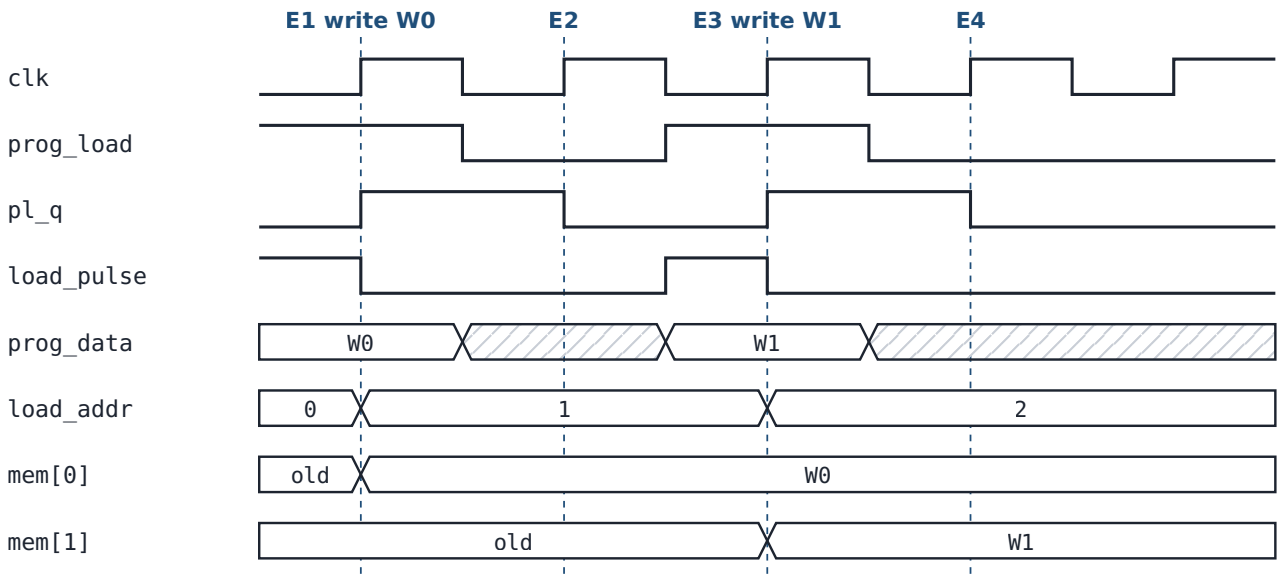


Figure 2: loading two words in Mode H with run=0. Inputs change while `clk` is low and are sampled at the next rising edge. `load_pulse` is combinational; the writes occur at E1 and E3.

7.3 Run, pause, halt and reset

Priority	Condition at an edge	Action
1	rst_n = 0	Clear acc, pc, ir, halted, load_addr. Memory unchanged. Load pulses are ignored. `pl_q` still samples `prog_load`.
2	run = 0 and load pulse	Write `mem[load_addr]`, increment `load_addr`. PC, ACC, IR and halted unchanged; a halted CPU stays halted.
3	run = 1 and halted = 0	Commit one instruction (7.1).
4	run = 1 and halted = 1	No change. `prog_load` is ignored.
5	run = 0, no load pulse	No change (pause). Execution resumes from the held PC when run returns to 1.
–	`prog_load` rising while run = 1	Ignored: memory unchanged. `pl_q` still follows `prog_load`.

7.4 Output behaviour and reading the 16-bit accumulator

`acc_out = byte_sel ? ACC[15:8] : ACC[7:0]`, a multiplexer on the live accumulator with no output register. It is 0x00 for both settings of `byte_sel` after reset. ACC changes only at an edge that commits LOAD, ADD, SUB, SHL or XOR, or on reset. `byte_sel` can change at any time and `acc_out` follows after the multiplexer delay, with no clock edge needed. This rule applies to RTL simulation, testbench and board.

A 16-bit value needs two reads, and ACC shall not change between them:

Step	byte_sel	acc_out	Condition
1	0	ACC[7:0] (low byte)	ACC stable
2	1	ACC[15:8] (high byte)	Same ACC value as step 1

Take both samples while ACC is not changing: with run=0, after HALT, or between host-driven clock pulses (Mode H). Sampling the two bytes while the program runs free can combine bytes of two different ACC values.

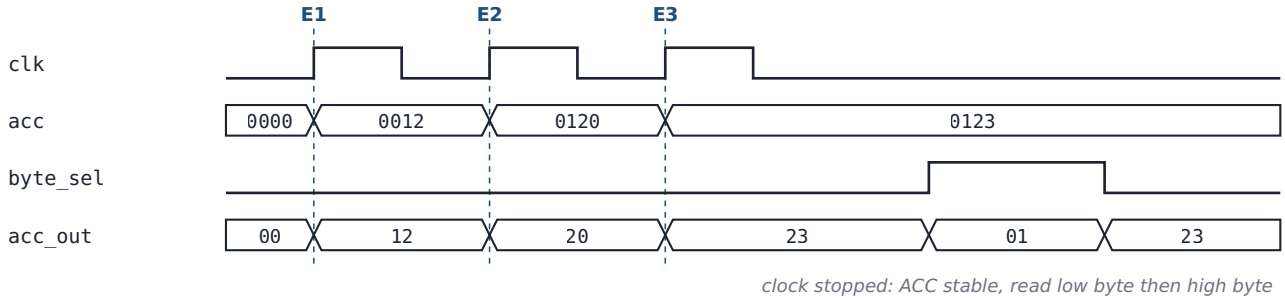


Figure 3: `acc_out` follows ACC and `byte_sel`. Values are those of the Section 8.1 program after E1 (LOAD 18), E2 (SHL 4) and E3 (ADD 3). With the clock stopped, the host reads the low byte (0x23), raises `byte_sel` and reads the high byte (0x01). `byte_sel` changes without a clock edge.

7.5 Edge cases

Case	Behaviour
PC = 7 executes a non-jump instruction	PC wraps to 0 and execution continues. This is not a halt.
ADD / SUB overflow or underflow	Wraps modulo 65536; carry and borrow cross the byte boundary. No flag.
SHL shifts significant bits out of bit 15	Those bits are lost. No flag, no rotate. idk how to implement this one
SHL 0, XOR 0	Instruction commits: PC + 1, IR updated. ACC unchanged.
run deasserted mid-program	State holds; resume continues from the held PC.
rst_n asserted mid-program	State registers cleared (7.3); program retained and can be re-run without reloading. All 16 bits of ACC cleared.
rst_n asserted during a load sequence	Words already written are kept; the next word goes to address 0.
`prog_load` high when reset is released	No write: `pl_q` sampled 1 during reset, so no rising edge is seen.
run = 1 at the edge where a load pulse would occur	Word not written; `load_addr` unchanged.
Load pulse while halted (run = 0)	Word written, `load_addr` increments; CPU stays halted.
SHL/JMP/JZ with operand[4:3] ≠ 0	Ignored; shift amount and target are operand[2:0].
HALT with non-zero operand	Operand ignored.
Execution of a word never loaded	Undefined (simulation shows X; silicon executes whatever the flops power up to). Programs shall load all 8 words.
9th program word loaded	Overwrites word 0.
`byte_sel` changes while ACC is changing	`acc_out` shows the selected byte of the current ACC; a host reading both bytes across a commit edge gets an incoherent value (7.4).

7.6 Instruction timing

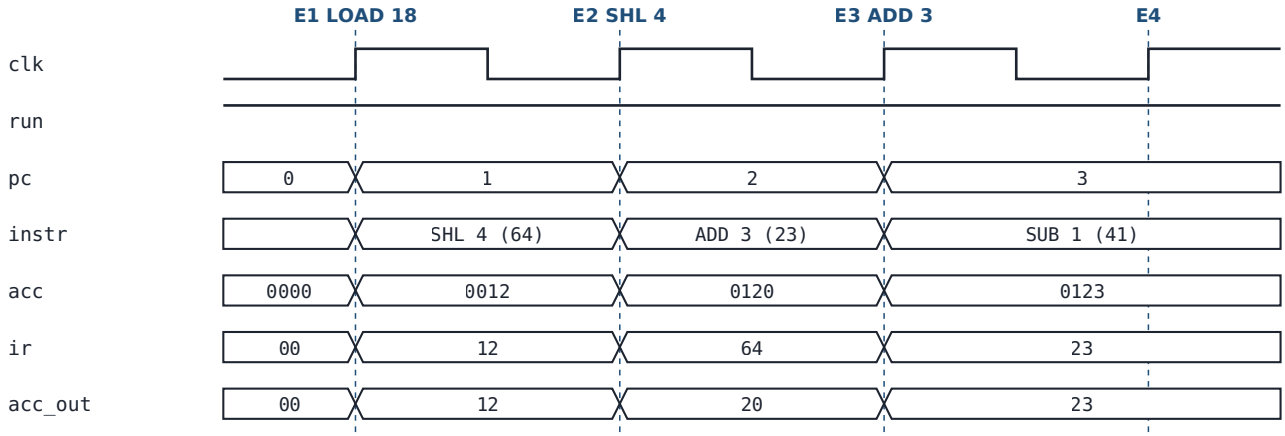


Figure 4: the first three instructions of the Section 8.1 program, `byte_sel` = 0. `instr` is combinational from `pc`; every committed result appears after the edge. `acc_out` follows ACC with no extra cycle.

8. Worked examples: programs

8.1 Building 0x0123 and counting down by three

Builds 0x0123 with the recipe of 6.6, subtracts 1 three times, and halts. Final ACC = 0x0120: low byte 0x20 (byte_sel=0), high byte 0x01 (byte_sel=1).

Addr	Mnemonic	Binary	Hex
0	LOAD 18	000_10010	0x12
1	SHL 4	011_00100	0x64
2	ADD 3	001_00011	0x23
3	SUB 1	010_00001	0x41
4	SUB 1	010_00001	0x41
5	SUB 1	010_00001	0x41
6	HALT	111_00000	0xE0
7	HALT (filler)	111_00000	0xE0

Cycle	PC	Instruction	ACC after	High byte (sel=1)	Low byte (sel=0)	PC after
1	0	LOAD 18	0x0012	0x00	0x12	1
2	1	SHL 4	0x0120	0x01	0x20	2
3	2	ADD 3	0x0123	0x01	0x23	3
4	3	SUB 1	0x0122	0x01	0x22	4
5	4	SUB 1	0x0121	0x01	0x21	5
6	5	SUB 1	0x0120	0x01	0x20	6
7	6	HALT	0x0120	0x01	0x20	6 (frozen)
8	6	HALT	0x0120	0x01	0x20	6 (frozen)

Cycle 7 executes HALT; cycle 8 and all later cycles are identical. 18 is the upper part of 0x0123 (0x0123 >> 4 = 0x12); SHL 4 moves it to 0x0120 and ADD 3 supplies the low nibble.

8.2 Countdown loop

Counts ACC down from 3 to 0 and halts. JZ is not taken twice and taken once.

Addr	Mnemonic	Binary	Hex
0	LOAD 3	000_00011	0x03
1	SUB 1	010_00001	0x41
2	JZ 4	110_00100	0xC4
3	JMP 1	101_00001	0xA1
4	HALT	111_00000	0xE0
5–7	HALT (filler)	111_00000	0xE0

Cycle	PC	Instruction	ACC after	PC after
1	0	LOAD 3	0x0003	1
2	1	SUB 1	0x0002	2
3	2	JZ 4 (not taken)	0x0002	3
4	3	JMP 1	0x0002	1
5	1	SUB 1	0x0001	2
6	2	JZ 4 (not taken)	0x0001	3
7	3	JMP 1	0x0001	1
8	1	SUB 1	0x0000	2
9	2	JZ 4 (taken)	0x0000	4
10	4	HALT	0x0000	4 (frozen)

Cycle 9 is the only taken JZ: SUB at cycle 8 made ACC zero and JZ sees that value (6.3).

9. Features to be verified

Every F item shall be demonstrated by RTL simulation; testbench tests are mapped to these identifiers. Edge counts refer to synchronized inputs (2.1).

ID	Feature	Pass criterion	Ref
F01	Reset values	After reset: PC, ACC (16 bits), IR, halted = 0 and `acc_out` = 0x00 for both settings of `byte_sel`.	7.3
F02	Reset keeps program	After reset mid-run, re-running without reloading produces a cycle-identical trace of `pc`, `acc`, `ir`, `halted`.	7.3
F03	Reset clears load address	After reset, the first loaded word lands at address 0.	4.2
F04	Reset clears halt	A halted CPU executes again after reset.	7.3
F05	Load edge detect	`prog_load` held high for 5 edges writes exactly one word.	7.2
F06	Sequential load	The i-th pulse writes word i, for i = 0..7.	7.2
F07	Load wrap	9th load overwrites word 0; words 1..7 unchanged.	7.2
F08	Load blocked while running	`prog_load` pulses with run=1 leave memory and `load_addr` unchanged.	7.3
F09	LOAD	ACC = zero-extended imm for imm = 0..31; upper 11 bits cleared when ACC was non-zero before.	6.2
F10	ADD	ACC = ACC + imm.	6.2
F11	ADD carry across bytes	A carry out of bit 7 increments the high byte.	6.4 #2
F12	ADD wrap	Result modulo 65536, no other side effect.	6.5
F13	SUB	ACC = ACC − imm.	6.2
F14	SUB borrow across bytes	A borrow into bit 8 decrements the high byte.	6.4 #3
F15	SUB wrap	Underflow gives the modulo-65536 result.	6.5
F16	SHL amounts	ACC << sh is correct for every sh = 0..7.	6.4 #4
F17	SHL loses bits	Bits shifted out of bit 15 are discarded, not rotated.	6.4 #5
F18	SHL ignored bits	operand[4:3] of SHL have no effect (all four values tested).	6.3
F19	XOR	ACC = ACC ^ imm.	6.2
F20	XOR zero-extension	XOR leaves the upper 11 bits of ACC unchanged.	6.4 #6
F21	Byte select	`byte_sel`=0 gives ACC[7:0] and 1 gives ACC[15:8] for every ACC value produced in F09–F20 and in the Section 8 programs.	7.4
F22	Byte select is combinational	`acc_out` follows `byte_sel` with no clock edge in between.	7.4
F23	No output register	`acc_out` shows the new ACC value immediately after the committing edge; no extra cycle of delay.	7.4
F24	JMP	All 8 targets reached.	6.2
F25	Ignored fields	operand[4:3] of JMP/JZ and the whole operand of HALT have no effect.	6.3
F26	JZ taken	ACC = 0: PC = target.	6.4 #8
F27	JZ not taken	ACC ≠ 0: PC = PC + 1.	6.4 #9
F28	JZ tests all 16 bits	Not taken for 0x0001, 0x0100, 0x8000, and every single-bit value 2^k, k = 0..15.	6.3

ID	Feature	Pass criterion	Ref
F29	JZ sees previous result	SUB followed immediately by JZ tests the SUB result.	6.3
F30	HALT freeze	PC, ACC, IR and halted constant for at least 8 edges after HALT with run=1, and across run toggles.	6.4 #10
F31	HALT sticky	Toggling run or rewriting memory while halted does not resume execution; only reset does.	7.3
F32	Pause	With run=0 no instruction commits and PC, ACC, IR, halted are unchanged.	7.3
F33	Resume	Execution continues from the held PC.	7.3
F34	PC wrap	PC 7 → 0 without halting.	7.5
F35	Single-cycle	Exactly one instruction commits per edge while run=1. Cycle-exact match with the reference model (7.1) over all F tests and at least 1000 random fully-loaded programs of 64 edges each with random run toggles.	7.1
F36	IR content	`ir` equals the last committed instruction.	4.2
F37	Port interface	Top-level port names, widths and directions match 3.1.	3.1
F38	No X	No X/Z on `acc_out` or architectural registers once reset and a full program load are done.	7.1
F39	Run with load pulse	run=1 at the edge where a load pulse would occur: word not written, `load_addr` unchanged.	7.5
F40	Load across reset release	`prog_load` high before and after reset release: no word written.	7.5
F41	Load while halted	Word written and `load_addr` increments; PC, ACC, IR unchanged; halted stays 1.	7.3
F42	Reset during load	After reset mid-sequence, the next word goes to address 0; earlier words retained.	7.5
F43	SHL 0 / XOR 0	Instruction commits (PC + 1, IR updated); ACC unchanged.	6.3
F44	Synchronizer latency (Mode O only)	Each of `run`, `prog_load`, `rst_n` takes effect exactly 2 edges later at the pins than in Mode H.	2.1
N1	Lint	Verilator `--lint-only -Wall` clean; every waiver justified in a comment.	10
N2	Synthesis	No latches, no multiplier or divider cells; flop count as accepted in 4.3.	2.2, 4.3
N3	Timing	Setup and hold slack ≥ 0 at the declared clock period and every declared corner, with the constraints of 2.3.	2.3
N4	Gate-level sim (optional)	Section 8.1 program on the netlist: after HALT, `acc_out` = 0x20 with byte_sel=0 and 0x01 with byte_sel=1. Checked through `acc_out` only.	3.3

10. Implementation guidance, deliverables and acceptance

10.1 Coding guidance

- One combinational block computes next-state values: defaults first, then `case (opcode)` with a default.
- Clocked blocks: one for architectural registers with synchronous reset; one for the program memory with no reset; one for the synchronizers (Mode O).
- Write constants with explicit widths. Zero-extend the immediate explicitly (`{11'b0, imm}`); never sign-extend it.
- Implement SHL with << and a 3-bit amount, or an equivalent three-level multiplexer shifter.
- Non-blocking assignments in clocked blocks, blocking assignments in combinational blocks.
- Drive `acc_out` with a continuous assignment from `acc` and `byte_sel`; do not register it.

10.2 Deliverables

- RTL source (`toy_cpu16.v` or equivalent) with the register names of 3.3.
- Testbench and reference model source. If a provided testbench is used, its version.
- Simulation log showing every applicable F test passing.
- Lint report (N1), synthesis report with flop count (N2), STA reports for each declared corner (N3).
- Design note containing:
  1. Clocking mode, SYNC_STAGES, clock period and the reason for the period.
  2. Harness, port-to-pin mapping, area allocation and reported utilisation (2.4).
  3. STA corners, input/output delay constraints, treatment of the `byte_sel` → `acc_out` path, and worst setup/hold slack per corner (2.3).
  4. Flop count, with every flop beyond 88 (94 in Mode O) identified (4.3).
  5. Tool versions (simulator, Verilator, Yosys, place-and-route flow) and the SKY130 PDK version or commit.
  6. Design explanation: loader edge detection with the timing of `pl_q`; why exactly one instruction commits per edge; why `acc_out` is a combinational byte multiplexer and the coherent-read rule; how carry and borrow cross bit 7/8; why JZ tests all 16 bits.
  7. Deviations from this document, each with justification, including any area lever applied. "None" if there are none.

10.3 Acceptance criteria

The design is accepted when F01–F43 pass (and F44 in Mode O), N1 and N2 are clean, N3 is met, and the design note contains items a to g.

11. Programming model, out of scope and risks

11.1 Programming model

- Programs are hand-assembled machine code: up to 8 instruction words (opcode + operand), as in Sections 6.4 and 8.
- The host pushes each byte onto `prog_data`, pulses `prog_load` (7.2), repeats for all 8 words, raises `run` and clocks the design. It reads the result with byte_sel=0 then byte_sel=1 (7.4).
- A cross-assembler script on a PC or on the host may convert mnemonics (LOAD 5, ADD 3) into bytes. Nothing executes on the CPU except loaded machine code.
- Bring-up: load all 8 words, set run=1, single-step with the host-driven clock (Mode H) or pause with run=0 (Mode O), and at each checkpoint sample `acc_out` with byte_sel=0 then 1. The reconstructed 16-bit trace shall match RTL simulation exactly.
- Capacity: 8 words in total, including HALT. A worst-case constant (5 instructions) leaves 2 words.
- Completion is not visible on a port: `halted` is internal. The host determines completion by clocking for the number of edges established in RTL simulation (7 for the Section 8.1 program), or by single-stepping (Mode H) and comparing both bytes of `acc_out` after each edge with the simulated trace.

11.2 Out of scope

- No register file (single accumulator only). Adding general-purpose registers requires re-checking the area budget (4.3).
- No interrupts, no pipelining, no multiply or divide instructions.
- No writable data memory beyond the accumulator.
- No condition flags; JZ is the only conditional.
- No program-memory readback and no halted status port.
- No debouncing of host inputs.
- No 32-bit variant.

11.3 Risk register

Risk	Likelihood	Impact	Mitigation
Post-synthesis area exceeds the declared allocation (2.4). Reference synthesis: 513 generic cells, shifter 86.	Medium	High: blocks submission	Synthesize early against the allocation; apply the levers of 4.3 in order (PROG_WORDS to 4, then remove XOR).
Mode H declared but host firmware changes inputs near the rising edge	Medium	High: partial instruction commit	If the host cannot guarantee that inputs change only while `clk` is low, declare Mode O (SYNC_STAGES=2).
Program loaded incorrectly (no readback)	Medium	Medium: wrong results on board	After loading, run a known program (Section 8.1) and check both bytes of `acc_out` before running the target program; reload on mismatch.
Wide-constant sequence hand-assembled wrongly	Medium	Low–Medium: wrong constants, not a hardware bug	Check every sequence in RTL simulation before board use (6.6).
Bytes read while ACC is changing	Low	Low: incoherent 16-bit read	Sample both bytes only with run=0, after HALT, or between host-driven clock pulses (7.4).